Déploiement d'Ollama + Open WebUI sur Kubernetes

Mamadou Falilou Diaw
Classe : DIC2-GIT
École Polytechnique de Thiès

1. Architecture

Deux services dans le namespace `llm` : **Ollama** (héberge les LLMs, API HTTP port 11434) et **Open WebUI** (interface de chat, port 8080). Sous Docker Compose ils communiquent via un réseau bridge et la résolution par nom de conteneur ; sous Kubernetes, un Service `ClusterIP` joue ce rôle et fournit un DNS stable (`ollama`) quel que soit le pod/IP courant.

2. Rôle de chaque ressource Kubernetes

Ressource	**Rôle**
Namespace (`llm`)	Isole les objets de l'app ; périmètre des NetworkPolicies.
ConfigMap	Injecte `OLLAMA_BASE_URL` sans le coder en dur dans l'image.
PVC `ollama-data`	Persiste les poids des modèles téléchargés.
PVC `open-webui-data`	Persiste historique de chat, comptes, réglages (SQLite).
Deployment ollama	Gère le pod ; stratégie `Recreate` évite deux pods sur le PVC RWO.
Deployment open-webui	Gère le pod ; init container attend que l'API ollama réponde.
Service ollama	DNS/IP stable (`ClusterIP`) pour que webui joigne ollama.
Service open-webui	Expose l'UI en interne et via `NodePort` fixe (31000).
Ingress	Route le trafic externe (par host) vers le Service open-webui.
NetworkPolicy*	Deny par défaut ; autorise seulement webui→ollama et ingress→webui.
PodDisruptionBudget*	Borne les évictions volontaires (drain, rollout).
HorizontalPodAutoscaler*	Scale open-webui 2–5 replicas sur charge CPU/mémoire.

*Ressources bonus, ajoutées au-delà du socle demandé.

3. Communication entre services

`open-webui` lit `OLLAMA_BASE_URL=http://ollama:11434` depuis son ConfigMap et appelle cette URL pour chaque complétion ; le DNS interne résout `ollama` vers le ClusterIP du Service, qui route vers le pod portant le label `app: ollama` (un seul actuellement : le PVC est `ReadWriteOnce` et le modèle vit en mémoire dans ce pod). Les utilisateurs externes atteignent `open-webui` via l'Ingress (routage par host) ou directement via le NodePort 31000.

4. Pourquoi des PersistentVolumeClaims

Un pod est éphémère : crash, drain de nœud ou rolling update détruisent tout ce qui n'est pas sur un volume monté. Sans PVC, chaque redémarrage re-téléchargerait les poids des modèles et effacerait tout l'historique de chat. Le PVC découple cet état du cycle de vie du pod : la Deployment peut recréer le pod n'importe où, il retrouve ses données.

5. Rôle des probes startup, readiness, liveness

Startup tolère un démarrage lent (pull d'image, init du dossier modèles, migrations DB) sans affaiblir les deux autres probes en régime stable. **Readiness** contrôle si le pod reçoit du trafic du Service — évite d'envoyer des requêtes à un pod encore en démarrage. **Liveness** détecte un pod bloqué (ex. sous-processus `llama-server` figé) et déclenche un redémarrage automatique, ce que readiness seul ne fait pas.

6. Docker Compose vs Kubernetes

	Docker Compose	**Kubernetes**
Portée	Une seule machine	Cluster de nœuds
Ordonnancement	Aucun	Scheduler place les pods sur un nœud dispo
Auto-guérison	`restart: unless-stopped`	Contrôleur réconcilie en continu l'état désiré
Scaling	Manuel, limité à l'hôte	Déclaratif (replicas), HPA automatisable
Réseau	Bridge, DNS par nom de conteneur	Service DNS cluster-wide, NetworkPolicy
Config	`.env` / variables inline	ConfigMap/Secret, découplés de l'image
Déploiements	Recréation manuelle des conteneurs	Rolling updates, PDB protège la dispo

7. Bonus implémentés

Kustomize (overlays `development`/`production`), Helm chart (`helm/llm-stack`), NetworkPolicy, PodDisruptionBudget, HPA (§2), et un pipeline GitHub Actions (lint, validation Kustomize/Helm, build image, déploiement de test sur cluster `kind` éphémère à chaque push).

8. Captures d'écran

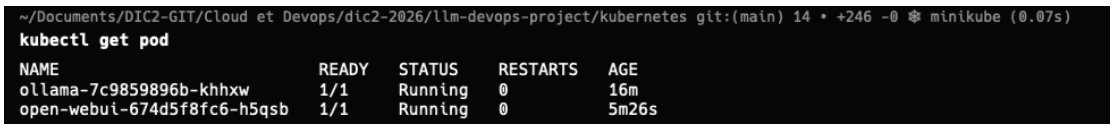


FIGURE 1 – Pods du namespace `llm` à l'état `Running`.

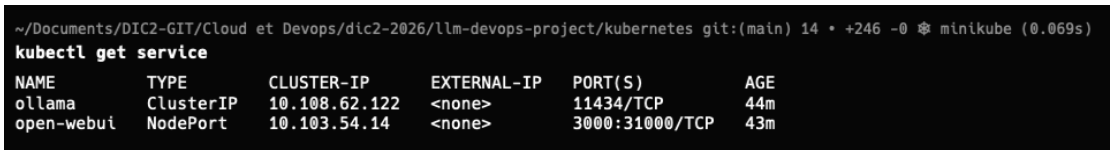


FIGURE 2 – Services exposant Ollama et Open WebUI.



FIGURE 3 – Page d'accueil d'Open WebUI.



FIGURE 4 – Réponse réussie de `llama3.2:3b`, `mistral:7b`, `deepseek-r1:8b`.